9 Using PyTorch to fight cancer

This chapter covers

- Breaking a large problem into smaller, easier ones
- Exploring the constraints of an intricate deep learning problem, and deciding on a structure and approach
- Downloading the training data

We have two main goals for this chapter. We'll start by covering the overall plan for part 2 of the book so that we have a solid idea of the larger scope the following individual chapters will be building toward. In chapter 10, we will begin to build out the data-parsing and data-manipulation routines that will produce data to be consumed in chapter 11 while training our first model. In order to do what's needed for those upcoming chapters well, we'll also use this chapter to cover some of the context in which our project will be operating: we'll go over data formats, data sources, and exploring the constraints that our problem domain places on us. Get used to performing these tasks, since you'll have to do them for any serious deep learning project!

9.1 Introduction to the use case

Our goal for this part of the book is to give you the tools to deal with situations where things aren't working, which is a far more common state of affairs than part 1 might have led you to believe. We can't predict every failure case or cover every debugging technique, but hopefully we'll give you enough to not feel stuck when you encounter a new roadblock. Similarly, we want to help you avoid situations with your own projects where you have no idea what you could do next when your projects are under-performing. Instead, we hope your ideas list will be so long that the challenge will be to prioritize!

In order to present these ideas and techniques, we need a context with some nuance and a fair bit of heft to it. We've chosen automatic detection of malignant tumors in the lungs using only a CT scan of a patient's chest as input. We'll be focusing on the technical challenges rather than the human impact, but make no mistake—even from just an engineering perspective, part 2 will require a more serious, structured approach than we needed in part 1 in order to have the project succeed.

NOTE CT scans are essentially 3D X-rays, represented as a 3D array of single-channel data. We'll cover them in more detail soon.

As you might have guessed, the title of this chapter is more eye-catching, implied hyperbole than anything approaching a serious statement of intent. Let us be precise: our project in this part of the book will take three-dimensional CT scans of human torsos as input and produce as output the location of suspected malignant tumors, if any exist.

Detecting lung cancer early has a huge impact on survival rate, but is difficult to do manually, especially in any comprehensive, whole-population sense. Currently, the work of reviewing the data must be performed by highly trained specialists, requires painstaking attention to detail, and it is dominated by cases where no cancer exists.

Doing that job well is akin to being placed in front of 100 haystacks and being told, "Determine which of these, if any, contain a needle." Searching this way results in the potential for missed warning signs, particularly in the early stages when the hints are more subtle. The human brain just isn't built well for that kind of monotonous work. And that, of course, is where deep learning comes in.

Automating this process is going to give us experience working in an uncooperative environment where we have to do more work from scratch, and there are fewer easy answers to problems that we might run into. Together, we'll get there, though! Once you're finished reading part 2, we think you'll be ready to start working on a real-world, unsolved problem of your own choosing.

We chose this problem of lung tumor detection for a few reasons. The primary reason is that the problem itself is unsolved! This is important, because we want to make it clear that you can use PyTorch to tackle cutting-edge projects effectively. We hope that increases your confidence in PyTorch as a framework, as well as in yourself as a developer. Another nice aspect of this problem space is that while it's unsolved, a lot of teams have been paying attention to it recently and have seen promising results. That means this challenge is probably right at the edge of our collective ability to solve; we won't be wasting our time on a problem that's actually decades away from

reasonable solutions. That attention on the problem has also resulted in a lot of high-quality papers and open source projects, which are a great source of inspiration and ideas. This will be a huge help once we conclude part 2 of the book, if you are interested in continuing to improve on the solution we create. We'll provide some links to additional information in chapter 14.

This part of the book will remain focused on the problem of detecting lung tumors, but the skills we'll teach are general. Learning how to investigate, preprocess, and present your data for training is important no matter what project you're working on. While we'll be covering preprocessing in the specific context of lung tumors, the general idea is that *this is what you should be prepared to do* for your project to succeed. Similarly, setting up a training loop, getting the right performance metrics, and tying the project's models together into a final application are all general skills that we'll employ as we go through chapters 9 through 14.

> **NOTE** While the end result of part 2 will work, the output will not be accurate enough to use clinically. We're focusing on using this as a motivating example for *teaching PyTorch*, not on employing every last trick to solve the problem.

9.2 Preparing for a large-scale project

This project will build off of the foundational skills learned in part 1. In particular, the content covering model construction from chapter 8 will be directly relevant. Repeated convolutional layers followed by a resolution-reducing downsampling layer will still make up the majority of our model. We will use 3D data as input to our model, however. This is conceptually similar to the 2D image data used in the last few chapters of part 1, but we will not be able to rely on all of the 2D-specific tools available in the PyTorch ecosystem.

The main differences between the work we did with convolutional models in chapter 8 and what we'll do in part 2 are related to how much effort we put into things outside the model itself. In chapter 8, we used a provided, off-the-shelf dataset and did little data manipulation before feeding the data into a model for classification. Almost all of our time and attention were spent building the model itself, whereas now we're not even going to begin designing the first of our two model architectures until chapter 11. That is a direct consequence of having nonstandard data without prebuilt libraries ready to hand us training samples suitable to plug into a model. We'll have to learn about our data and implement quite a bit ourselves.

Even when that's done, this will not end up being a case where we convert the CT to a tensor, feed it into a neural network, and have the answer pop out the other side. As is common for real-world use cases such as this, a workable approach will be more complicated to account for confounding factors such as limited data availability, finite computational resources, and limitations on our ability to design effective models. Please keep that in mind as we build to a high-level explanation of our project architecture.

Speaking of finite computational resources, part 2 will require access to a GPU to achieve reasonable training speeds, preferably one with at least 8 GB of RAM. Trying

to train the models we will build on CPU could take weeks![1] If you don't have a GPU handy, we provide pretrained models in chapter 14; the nodule analysis script there can probably be run overnight. While we don't want to tie the book to proprietary services if we don't have to, we should note that at the time of writing, Colaboratory (https://colab.research.google.com) provides free GPU instances that might be of use. PyTorch even comes preinstalled! You will also need to have at least 220 GB of free disk space to store the raw training data, cached data, and trained models.

> **NOTE** Many of the code examples presented in part 2 have complicating details omitted. Rather than clutter the examples with logging, error handling, and edge cases, the text of this book contains only code that expresses the core idea under discussion. Full working code samples can be found on the book's website (www.manning.com/books/deep-learning-with-pytorch) and GitHub (https://github.com/deep-learning-with-pytorch/dlwpt-code).

OK, we've established that this is a hard, multifaceted problem, but what are we going to do about it? Instead of looking at an entire CT scan for signs of tumors or their potential malignancy, we're going to solve a series of simpler problems that will combine to provide the end-to-end result we're interested in. Like a factory assembly line, each step will take raw materials (data) and/or output from previous steps, perform some processing, and hand off the result to the next station down the line. Not every problem needs to be solved this way, but breaking off chunks of the problem to solve in isolation is often a great way to start. Even if it turns out to be the wrong approach for a given project, it's likely we'll have learned enough while working on the individual chunks that we'll have a good idea how to restructure our approach into something successful.

Before we get into the details of how we'll break down our problem, we need to learn some details about the medical domain. While the code listings will tell you *what* we're doing, learning about radiation oncology will explain *why*. Learning about the problem space is crucial, no matter what domain it is. Deep learning is powerful, but it's not magic, and trying to apply it blindly to nontrivial problems will likely fail. Instead, we have to combine insights into the space with intuition about neural network behavior. From there, disciplined experimentation and refinement should give us enough information to close in on a workable solution.

9.3 What is a CT scan, exactly?

Before we get too far into the project, we need to take a moment to explain what a CT scan is. We will be using data from CT scans extensively as the main data format for our project, so having a working understanding of the data format's strengths, weaknesses, and fundamental nature will be crucial to utilizing it well. The key point we noted earlier is this: CT scans are essentially 3D X-rays, represented as a 3D array of

[1] We presume—we haven't tried it, much less timed it.

single-channel data. As we might recall from chapter 4, this is like a stacked set of grayscale PNG images.

Voxel

A *voxel* is the 3D equivalent to the familiar two-dimensional pixel. It encloses a volume of space (hence, "volumetric pixel"), rather than an area, and is typically arranged in a 3D grid to represent a field of data. Each of those dimensions will have a measurable distance associated with it. Often, voxels are cubic, but for this chapter, we will be dealing with voxels that are rectangular prisms.

In addition to medical data, we can see similar voxel data in fluid simulations, 3D scene reconstructions from 2D images, light detection and ranging (LIDAR) data for self-driving cars, and many other problem spaces. Those spaces all have their individual quirks and subtleties, and while the APIs that we're going to cover here apply generally, we must also be aware of the nature of the data we're using with those APIs if we want to be effective.

Each voxel of a CT scan has a numeric value that roughly corresponds to the average mass density of the matter contained inside. Most visualizations of that data show high-density material like bones and metal implants as white, low-density air and lung tissue as black, and fat and tissue as various shades of gray. Again, this ends up looking somewhat similar to an X-ray, with some key differences.

The primary difference between CT scans and X-rays is that whereas an X-ray is a projection of 3D intensity (in this case, tissue and bone density) onto a 2D plane, a CT scan retains the third dimension of the data. This allows us to render the data in a variety of ways: for example, as a grayscale solid, which we can see in figure 9.1.

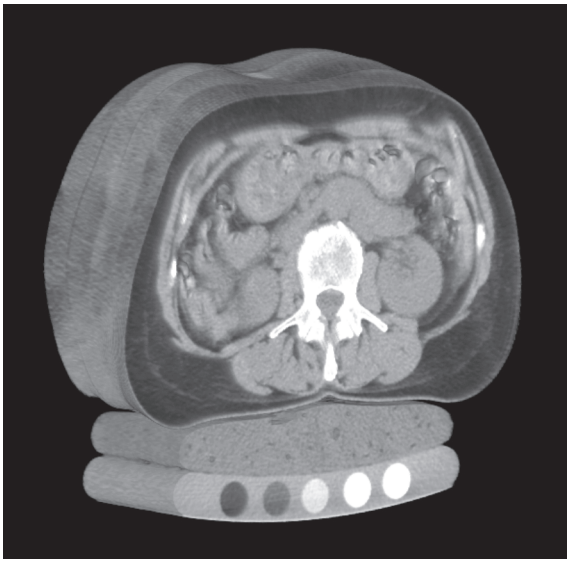


Figure 9.1 A CT scan of a human torso showing, from the top, skin, organs, spine, and patient support bed. Source: http://mng.bz/04r6; Mindways CT Software / CC BY-SA 3.0 (https://creativecommons.org/licenses/by-sa/3.0/deed.en).

NOTE CT scans actually measure radiodensity, which is a function of both mass density and atomic number of the material under examination. For our purposes here, the distinction isn't relevant, since the model will consume and learn from the CT data no matter what the exact units of the input happen to be.

This 3D representation also allows us to "see inside" the subject by hiding tissue types we are not interested in. For example, we can render the data in 3D and restrict visibility to only bone and lung tissue, as in figure 9.2.

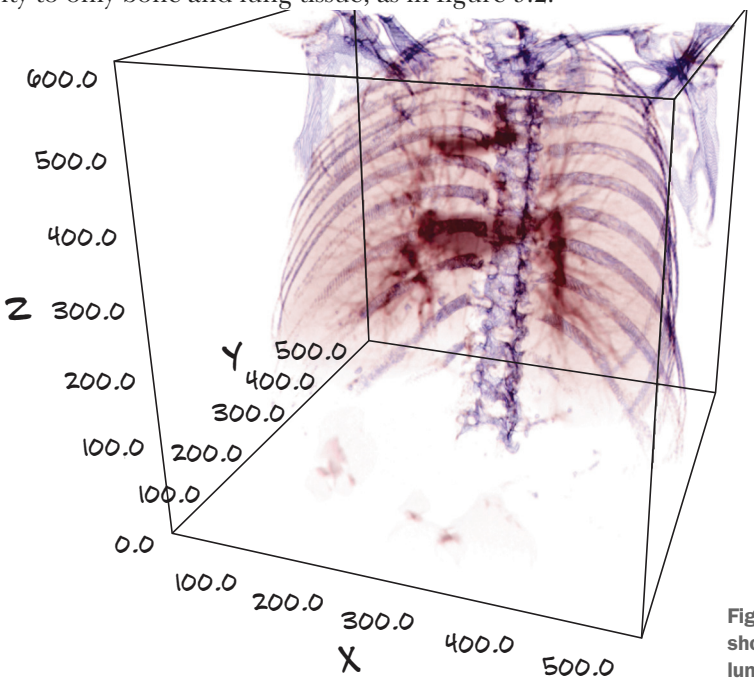


Figure 9.2 A CT scan showing ribs, spine, and lung structures

CT scans are much more difficult to acquire than X-rays, because doing so requires a machine like the one shown in figure 9.3 that typically costs upward of a million dollars new and requires trained staff to operate it. Most hospitals and some well-equipped clinics have a CT scanner, but they aren't nearly as ubiquitous as X-ray machines. This, combined with patient privacy regulations, can make it somewhat difficult to get CT scans unless someone has already done the work of gathering and organizing a collection of them.

Figure 9.3 also shows an example bounding box for the area contained in the CT scan. The bed the patient is resting on moves back and forth, allowing the scanner to image multiple slices of the patient and hence fill the bounding box. The scanner's darker, central ring is where the actual imaging equipment is located.

A final difference between a CT scan and an X-ray is that the data is a digital-only format. *CT* stands for *computed tomography* (https://en.wikipedia.org/wiki/CT_scan#Process).



Figure 9.3 A patient inside a CT scanner, with the CT scan's bounding box overlaid. Other than in stock photos, patients don't typically wear street clothes while in the machine.

The raw output of the scanning process doesn't look particularly meaningful to the human eye and must be properly reinterpreted by a computer into something we can understand. The settings of the CT scanner when the scan is taken can have a large impact on the resulting data.

While this information might not seem particularly relevant, we have actually learned something that is: from figure 9.3, we can see that the way the CT scanner measures distance along the head-to-foot axis is different than the other two axes. The patient actually moves along that axis! This explains (or at least is a strong hint as to) why our voxels might not be cubic, and also ties into how we approach massaging our data in chapter 12. This is a good example of why we need to understand our problem space if we're going to make effective choices about how to solve our problem. When starting to work on your own projects, be sure you do the same investigation into the details of your data.

9.4 The project: An end-to-end detector for lung cancer

Now that we've got our heads wrapped around the basics of CT scans, let's discuss the structure of our project. Most of the bytes on disk will be devoted to storing the CT scans' 3D arrays containing density information, and our models will primarily consume various subslices of those 3D arrays. We're going to use five main steps to go from examining a whole-chest CT scan to giving the patient a lung cancer diagnosis.

Our full, end-to-end solution shown in figure 9.4 will load CT data files to produce a `Ct` instance that contains the full 3D scan, combine that with a module that performs *segmentation* (flagging voxels of interest), and then group the interesting voxels into small lumps in the search for candidate *nodules*.

The nodule locations are combined back with the CT voxel data to produce nodule candidates, which can then be examined by our nodule classification model to determine whether they are actually nodules in the first place and, eventually, whether

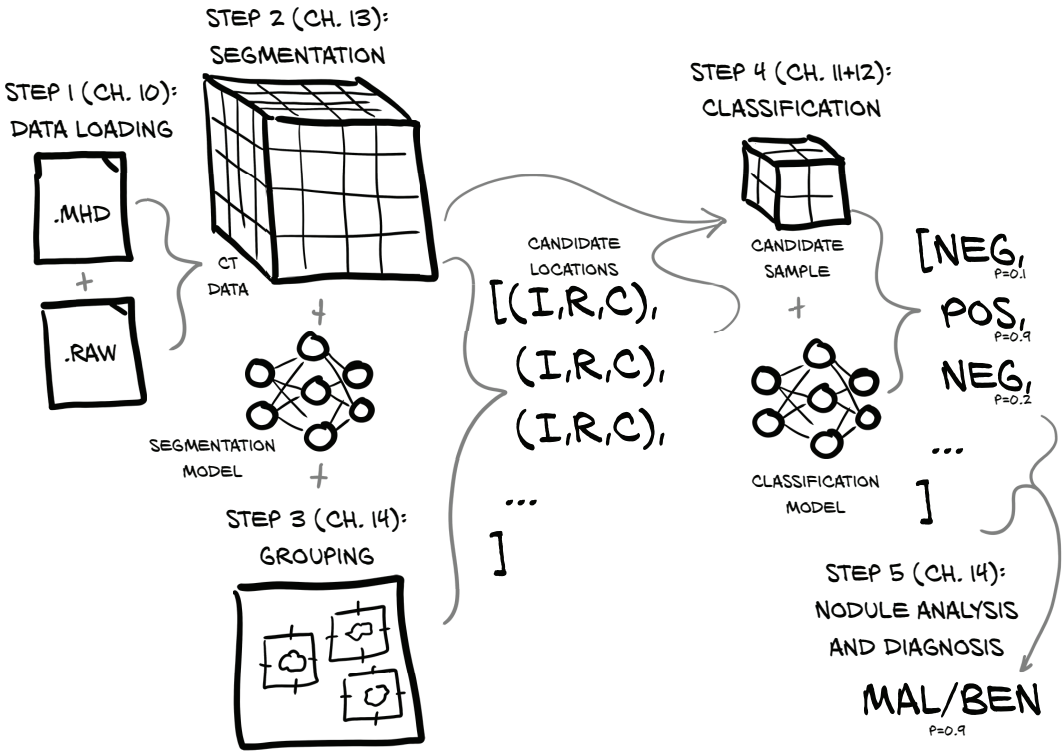


Figure 9.4 The end-to-end process of taking a full-chest CT scan and determining whether the patent has a malignant tumor

Nodules

A mass of tissue made of proliferating cells in the lung is a *tumor*. A tumor can be *benign* or it can be *malignant*, in which case it is also referred to as *cancer*. A small tumor in the lung (just a few millimeters wide) is called a *nodule*. About 40% of lung nodules turn out to be malignant—small cancers. It is very important to catch those as early as possible, and this depends on medical imaging of the kind we are looking at here.

they're malignant. This latter task is particularly difficult because malignancy might not be apparent from CT imaging alone, but we'll see how far we get. Last, each of those individual, per-nodule classifications can then be combined into a whole-patient diagnosis.

In more detail, we will do the following:

1. Load our raw CT scan data into a form that we can use with PyTorch. Putting raw data into a form usable by PyTorch will be the first step in any project you face. The process is somewhat less complicated with 2D image data and simpler still with non-image data.

2. Identify the voxels of potential tumors in the lungs using PyTorch to implement a technique known as *segmentation*. This is roughly akin to producing a heatmap of areas that should be fed into our classifier in step 3. This will allow us to focus on potential tumors inside the lungs and ignore huge swaths of uninteresting anatomy (a person can't have lung cancer in the stomach, for example).

   Generally, being able to focus on a single, small task is best while learning. With experience, there are some situations where more complicated model structures can yield superlative results (for example, the GAN game we saw in chapter 2), but designing those from scratch requires extensive mastery of the basic building blocks first. Gotta walk before you run, and all that.

3. Group interesting voxels into lumps: that is, candidate nodules (see figure 9.5 for more information on nodules). Here, we will find the rough center of each hotspot on our heatmap.

   Each nodule can be located by the index, row, and column of its center point. We do this to present a simple, constrained problem to the final classifier. Grouping voxels will not involve PyTorch directly, which is why we've pulled this out into a separate step. Often, when working with multistep solutions, there will be non-deep-learning glue steps between the larger, deep-learning-powered portions of the project.

4. Classify candidate nodules as actual nodules or non-nodules using 3D convolution.

   This will be similar in concept to the 2D convolution we covered in chapter 8. The features that determine the nature of a tumor from a candidate structure are local to the tumor in question, so this approach should provide a good balance between limiting input data size and excluding relevant information. Making scope-limiting decisions like this can keep each individual task constrained, which can help limit the amount of things to examine when troubleshooting.

5. Diagnose the patient using the combined per-nodule classifications.

   Similar to the nodule classifier in the previous step, we will attempt to determine whether the nodule is benign or malignant based on imaging data alone. We will take a simple maximum of the per-tumor malignancy predictions, as only one tumor needs to be malignant for a patient to have cancer. Other projects might want to use different ways of aggregating the per-instance predictions into a file score. Here, we are asking, "Is there anything suspicious?" so maximum is a good fit for aggregation. If we were looking for quantitative information like "the ratio of type A tissue to type B tissue," we might take an appropriate mean instead.

Figure 9.4 only depicts the final path through the system once we've built and trained all of the requisite models. The actual work required to train the relevant models will be detailed as we get closer to implementing each step.

The data we'll use for training provides human-annotated output for both steps 3 and 4. This allows us to treat steps 2 and 3 (identifying voxels and grouping them into nodule candidates) as almost a separate project from step 4 (nodule candidate

On the shoulders of giants

We are *standing on the shoulders of giants* when deciding on this five-step approach. We'll discuss these giants and their work more in chapter 14. There isn't any particular reason why we should know in advance that this project structure will work well for this problem; instead, we're relying on others who have actually implemented similar things and reported success when doing so. Expect to have to experiment to find workable approaches when transitioning to a different domain, but always try to learn from earlier efforts in the space and from those who have worked in similar areas and have discovered things that might transfer well. Go out there, look for what others have done, and use that as a benchmark. At the same time, avoid getting code and running it *blindly*, because you need to fully understand the code you're running in order to use the results to make progress for yourself.

classification). Human experts have annotated the data with nodule locations, so we can work on either steps 2 and 3 or step 4 in whatever order we prefer.

We will first work on step 1 (data loading), and then jump to step 4 before we come back and implement steps 2 and 3, since step 4 (classification) requires an approach similar to what we used in chapter 8, using multiple convolutional and pooling layers to aggregate spatial information before feeding it into a linear classifier. Once we've got a handle on our classification model, we can start working on step 2 (segmentation). Since segmentation is the more complicated topic, we want to tackle it without having to learn both segmentation and the fundamentals of CT scans and malignant tumors at the same time. Instead, we'll explore the cancer-detection space while working on a more familiar classification problem.

This approach of starting in the middle of the problem and working our way out probably seems odd. Starting at step 1 and working our way forward would make more intuitive sense. Being able to carve up the problem and work on steps independently is useful, however, since it can encourage more modular solutions; in addition, it's easier to partition the workload between members of a small team. Also, actual clinical users would likely prefer a system that flags suspicious nodules for review rather than provides a single binary diagnosis. Adapting our modular solution to different use cases will probably be easier than if we'd done a monolithic, from-the-top system.

As we work our way through implementing each step, we'll be going into a fair bit of detail about lung tumors, as well as presenting a lot of fine-grained detail about CT scans. While that might seem off-topic for a book that's focused on PyTorch, we're doing so specifically so that you begin to develop an intuition about the problem space. That's crucial to have, because the space of all possible solutions and approaches is too large to effectively code, train, and evaluate.

If we were working on a different project (say, the one you tackle after finishing this book), we'd still need to do an investigation to understand the data and problem space. Perhaps you're interested in satellite mapping, and your next project needs to consume pictures of our planet taken from orbit. You'd need to ask questions about the wavelengths being collected—do you get only normal RGB, or something more

exotic? What about infrared or ultraviolet? In addition, there might be impacts on the images based on time of day, or if the imaged location isn't directly under the satellite, skewing the image. Will the image need correction?

Even if your hypothetical *third* project's data type remains the same, it's probable that the domain you'll be working in will change things, possibly drastically. Processing camera output for self-driving cars still involves 2D images, but the complications and caveats are wildly different. For example, it's much less likely that a mapping satellite will need to worry about the sun shining into the camera, or getting mud on the lens!

We must be able to use our intuition to guide our investigation into potential optimizations and improvements. That's true of deep learning projects in general, and we'll practice using our intuition as we go through part 2. So, let's do that. Take a quick step back, and do a gut check. What does your intuition say about this approach? Does it seem overcomplicated to you?

9.4.1 Why can't we just throw data at a neural network until it works?

After reading the last section, we couldn't blame you for thinking, "This is nothing like chapter 8!" You might be wondering why we've got two separate model architectures or why the overall data flow is so complicated. Well, our approach is different from that in chapter 8 for a reason. It's a hard task to automate, and people haven't fully figured it out yet. That difficulty translates to complexity; once we as a society have solved this problem definitively, there will probably be an off-the-shelf library package we can grab to have it Just Work, but we're not there just yet.

Why so difficult, though?

Well, for starters, the majority of a CT scan is fundamentally uninteresting with regard to answering the question, "Does this patient have a malignant tumor?" This makes intuitive sense, since the vast majority of the patient's body will consist of healthy cells. In the cases where there is a malignant tumor, up to 99.9999% of the voxels in the CT still won't be cancer. That ratio is equivalent to a two-pixel blob of incorrectly tinted color somewhere on a high-definition television, or a single misspelled word out of a shelf of novels.

Can you identify the white dot in the three views of figure 9.5 that has been flagged as a nodule?[2]

If you need a hint, the index, row, and column values can be used to help find the relevant blob of dense tissue. Do you think you could figure out the relevant properties of tumors given only images (and that means *only* the images—no index, row, and column information!) like these? What if you were given the entire 3D scan, not just three slices that intersect the interesting part of the scan?

> **NOTE** Don't fret if you can't locate the tumor! We're trying to illustrate just how subtle this data can be—the fact that it is hard to identify visually is the entire point of this example.

[2]The `series_uid` of this sample is `1.3.6.1.4.1.14519.5.2.1.6279.6001.126264578931778258890371755354`, which can be useful if you'd like to look at it in detail later.

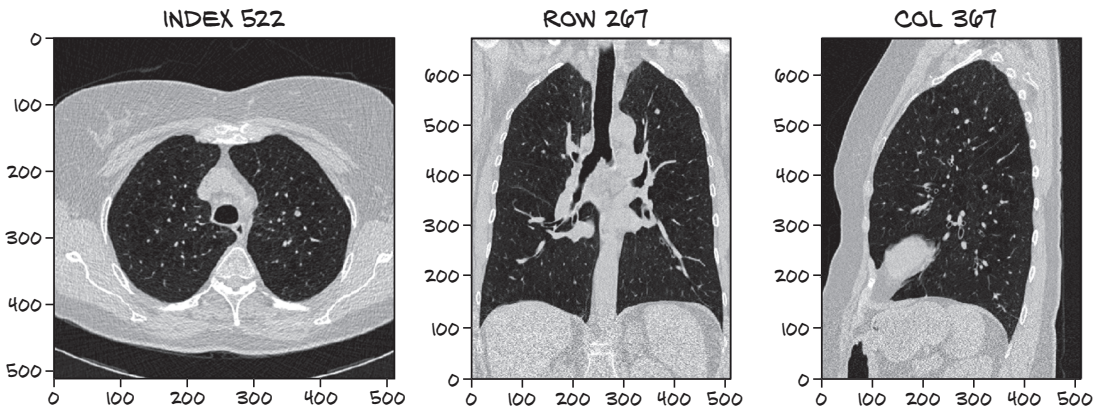


Figure 9.5 A CT scan with approximately 1,000 structures that look like tumors to the untrained eye. Exactly one has been identified as a nodule when reviewed by a human specialist. The rest are normal anatomical structures like blood vessels, lesions, and other non-problematic lumps.

You might have seen elsewhere that end-to-end approaches for detection and classification of objects are very successful in general vision tasks. TorchVision includes end-to-end models like Fast R-CNN/Mask R-CNN, but these are typically trained on hundreds of thousands of images, and those datasets aren't constrained by the number of samples from rare classes. The project architecture we will use has the benefit of working well with a more modest amount of data. So while it's certainly theoretically possible to just throw an arbitrarily large amount of data at a neural network until it learns the specifics of the proverbial lost needle, as well as how to ignore the hay, it's going to be practically prohibitive to collect enough data and wait for a long enough time to train the network properly. That won't be the *best* approach since the results are poor, and most readers won't have access to the compute resources to pull it off at all.

To come up with the best solution, we could investigate proven model designs that can better integrate data in an end-to-end manner.[3] These complicated designs are capable of producing high-quality results, but they're not the *best* because understanding the design decisions behind them requires having mastered fundamental concepts first. That makes these advanced models poor candidates to use while teaching those same fundamentals!

That's not to say that our multistep design is the *best* approach, either, but that's because "best" is only relative to the criteria we chose to evaluate approaches. There are *many* "best" approaches, just as there are many goals we could have in mind as we work on a project. Our self-contained, multistep approach has some disadvantages as well.

Recall the GAN game from chapter 2. There, we had two networks cooperating to produce convincing forgeries of old master artists. The artist would produce a candidate work, and the scholar would critique it, giving the artist feedback on how to

[3]For example, Retina U-Net (https://arxiv.org/pdf/1811.08661.pdf) and FishNet (http://mng.bz/K240).

improve. Put in technical terms, the structure of the model allowed gradients to backpropagate from the final classifier (fake or real) to the earliest parts of the project (the artist).

Our approach for solving the problem won't use end-to-end gradient backpropagation to directly optimize for our end goal. Instead, we'll optimize discrete chunks of the problem individually, since our segmentation model and classification model won't be trained in tandem with each other. That might limit the top-end effectiveness of our solution, but we feel that this will make for a much better learning experience.

We feel that being able to focus on a single step at a time allows us to zoom in and concentrate on the smaller number of new skills we're learning. Each of our two models will be focused on performing exactly one task. Similar to a human radiologist as they review slice after slice of CT, the job gets much easier to train for if the scope is well contained. We also want to provide tools that allow for rich manipulation of the data. Being able to zoom in and focus on the detail of a particular location will have a huge impact on overall productivity while training the model compared to having to look at the entire image at once. Our segmentation model is forced to consume the entire image, but we will structure things so that our classification model gets a zoomed-in view of the areas of interest.

Step 3 (grouping) will produce and step 4 (classification) will consume data similar to the image in figure 9.6 containing sequential transverse slices of a tumor. This image is a close-up view of a (potentially malignant, or at least indeterminate) tumor, and it is what we're going to train the step 4 model to identify, and the step 5 model to classify as either benign or malignant. While this lump may seem nondescript to an untrained eye (or untrained convolutional network), identifying the warning signs of malignancy in this sample is at least a far more constrained problem than having to consume the entire CT we saw earlier. Our code for the next chapter will provide routines to produce zoomed-in nodule images like figure 9.6.

We will perform the step 1 data-loading work in chapter 10, and chapters 11 and 12 will focus on solving the problem of classifying these nodules. After that, we'll back up to work on step 2 (using segmentation to find the candidate tumors) in chapter 13, and then we'll close out part 2 of the book in chapter 14 by implementing the end-to-end project with step 3 (grouping) and step 5 (nodule analysis and diagnosis).

NOTE Standard rendering of CTs places the superior at the top of the image (basically, the head goes up), but CTs order their slices such that the first slice is the inferior (toward the feet). So, Matplotlib renders the images upside down unless we take care to flip them. Since that flip doesn't really matter to our model, we won't complicate the code paths between our raw data and the model, but we will add a flip to our rendering code to get the images right-side up. For more information about CT coordinate systems, see section 10.4.

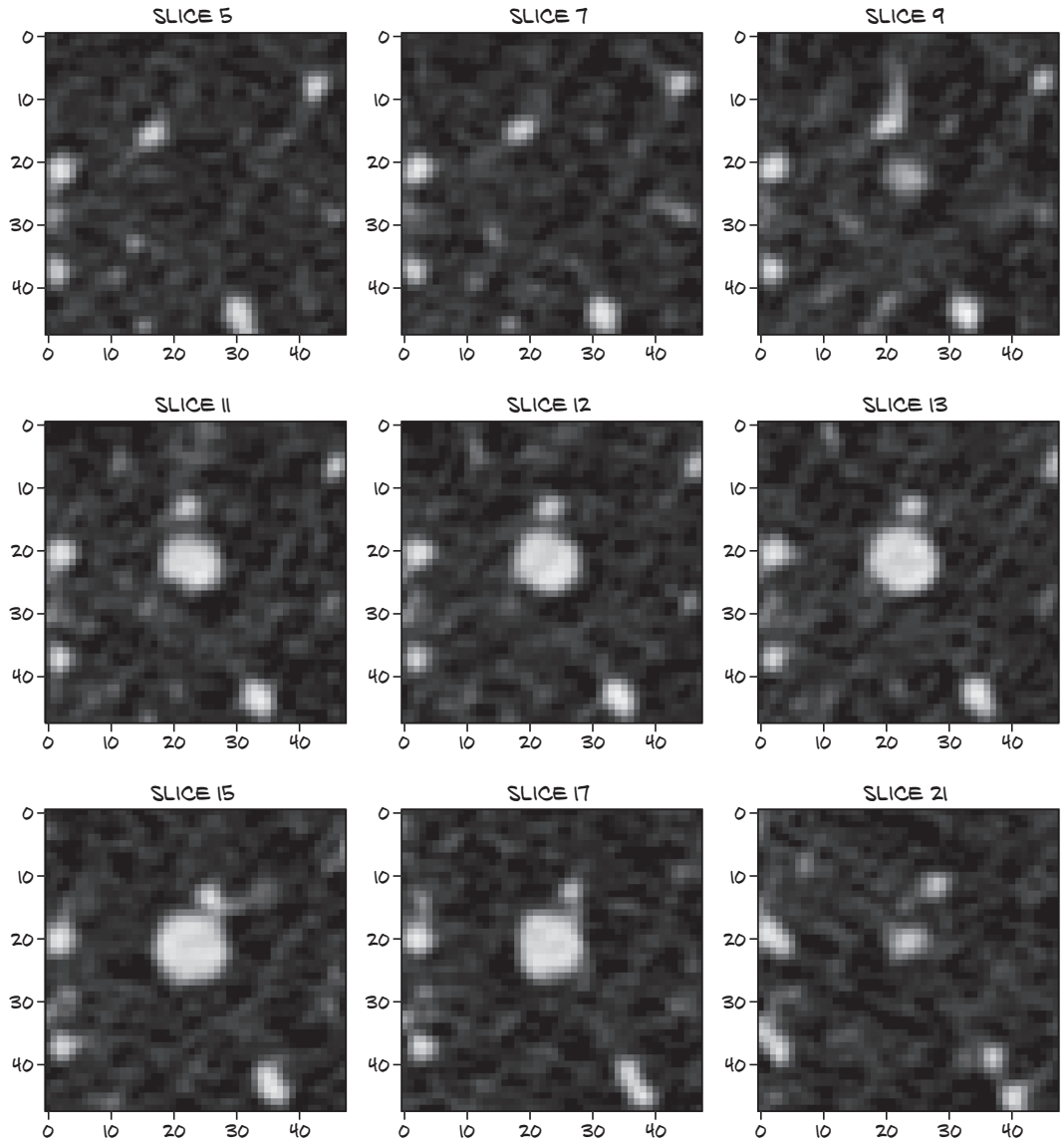


Figure 9.6 A close-up, multislice crop of the tumor from the CT scan in figure 9.5

Let's repeat our high-level overview in figure 9.7.

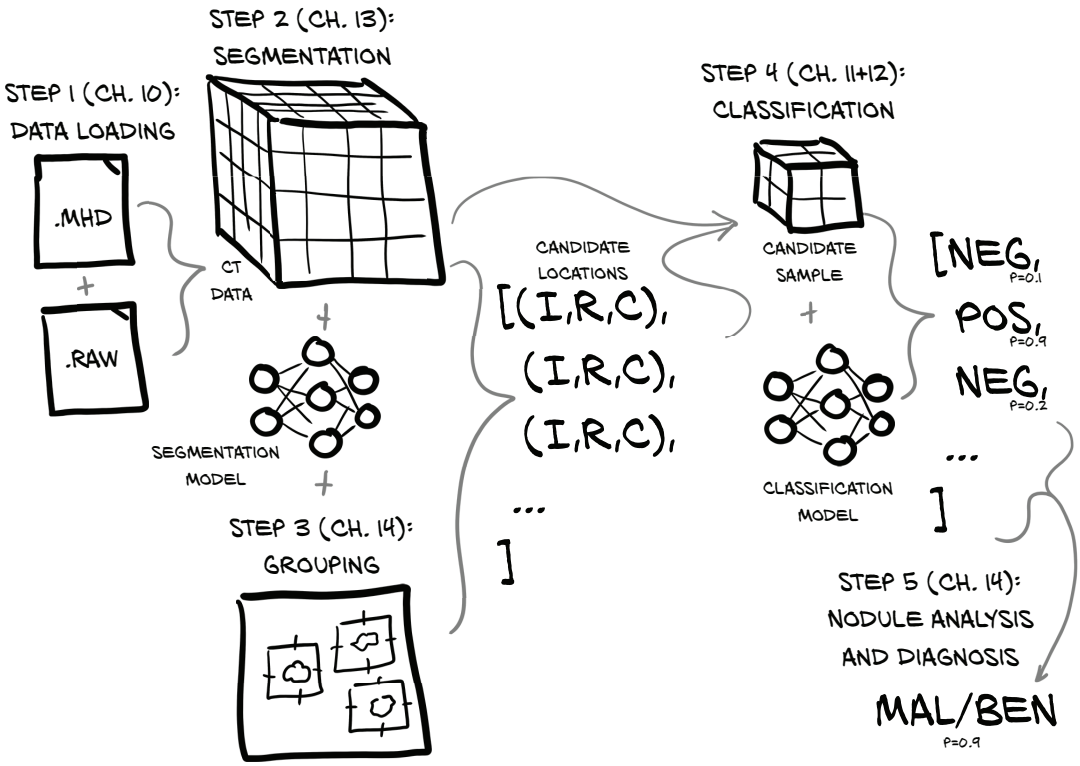


Figure 9.7 The end-to-end process of taking a full-chest CT scan and determining whether the patient has a malignant tumor

9.4.2 What is a nodule?

As we've said, in order to understand our data well enough to use it effectively, we need to learn some specifics about cancer and radiation oncology. One last key thing we need to understand is what a *nodule* is. Simply put, a nodule is any of the myriad lumps and bumps that might appear inside someone's lungs. Some are problematic from a health-of-the-patient perspective; some are not. The precise definition[4] limits the size of a nodule to 3 cm or less, with a larger lump being a *lung mass*; but we're going to use *nodule* interchangeably for all such anatomical structures, since it's a somewhat arbitrary cutoff and we're going to deal with lumps on both sides of 3 cm using the same code paths. A nodule—a small mass in the lung—can turn out to be benign or a malignant tumor (also referred to as *cancer*). From a radiological perspective, a nodule is really similar to other lumps that have a wide variety of causes: infection, inflammation, blood-supply issues, malformed blood vessels, and diseases other than tumors.

[4] Eric J. Olson, "Lung nodules: Can they be cancerous?" Mayo Clinic, http://mng.bz/yyge.

The key part is this: the cancers that we are trying to detect will *always* be nodules, either suspended in the very non-dense tissue of the lung or attached to the lung wall. That means we can limit our classifier to only nodules, rather than have it examine all tissue. Being able to restrict the scope of expected inputs will help our classifier learn the task at hand.

This is another example of how the underlying deep learning techniques we'll use are universal, but they can't be applied blindly.[5] We'll need to understand the field we're working in to make choices that will serve us well.

In figure 9.8, we can see a stereotypical example of a malignant nodule. The smallest nodules we'll be concerned with are only a few millimeters across, though the one in figure 9.8 is larger. As we discussed earlier in the chapter, this makes the smallest nodules approximately a million times smaller than the CT scan as a whole. More than half of the nodules detected in patients are not malignant.[6]

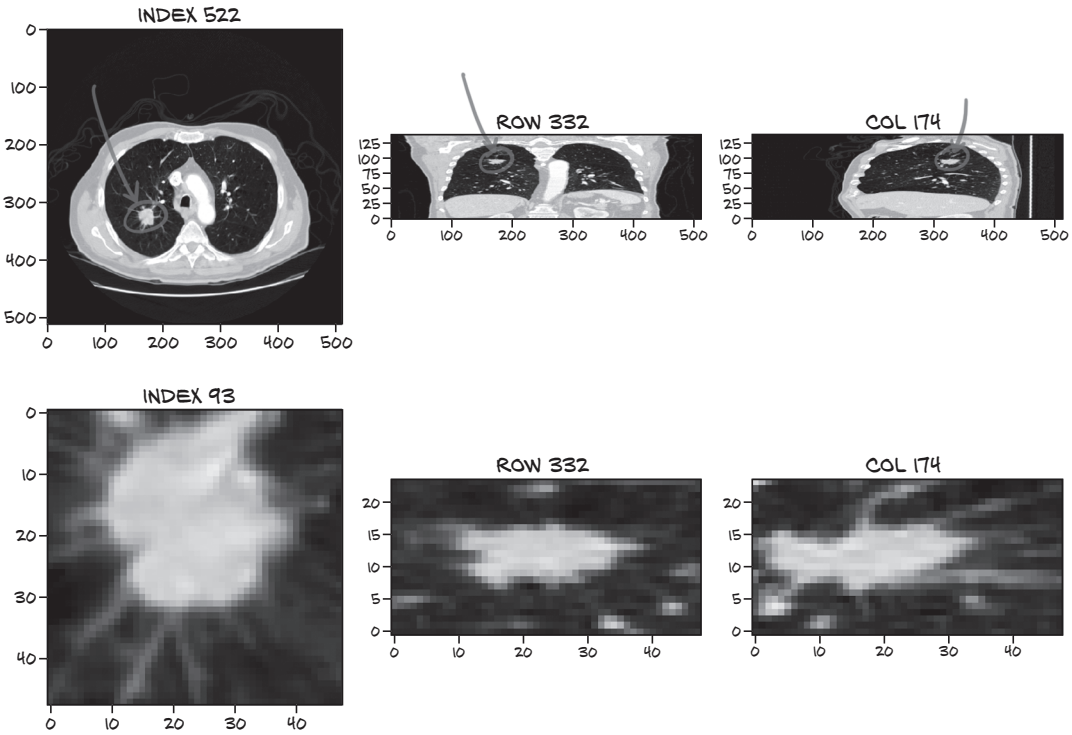


Figure 9.8 A CT scan with a malignant nodule displaying a visual discrepancy from other nodules

[5]Not if we want decent results, at least.

[6]According to the National Cancer Institute Dictionary of Cancer Terms: http://mng.bz/jgBP.

9.4.3 Our data source: The LUNA Grand Challenge

The CT scans we were just looking at come from the LUNA (LUng Nodule Analysis) Grand Challenge. The LUNA Grand Challenge is the combination of an open dataset with high-quality labels of patient CT scans (many with lung nodules) and a public ranking of classifiers against the data. There is something of a culture of publicly sharing medical datasets for research and analysis; open access to such data allows researchers to use, combine, and perform novel work on this data without having to enter into formal research agreements between institutions (obviously, some data is kept private as well). The goal of the LUNA Grand Challenge is to encourage improvements in nodule detection by making it easy for teams to compete for high positions on the leader board. A project team can test the efficacy of their detection methods against standardized criteria (the dataset provided). To be included in the public ranking, a team must provide a scientific paper describing the project architecture, training methods, and so on. This makes for a great resource to provide further ideas and inspiration for project improvements.

> **NOTE** Many CT scans "in the wild" are incredibly messy, in terms of idiosyncrasies between various scanners and processing programs. For example, some scanners indicate areas of the CT scan that are outside of the scanner's field of view by setting the density of those voxels to something negative. CT scans can also be acquired with a variety of settings on the CT scanner, which can change the resulting image in ways ranging from subtly to wildly different. Although the LUNA data is generally clean, be sure to check your assumptions if you incorporate other data sources.

We will be using the LUNA 2016 dataset. The LUNA site (https://luna16.grand-challenge.org/Description) describes two tracks for the challenge: the first track, "Nodule detection (NDET)," roughly corresponds to our step 1 (segmentation); and the second track, "False positive reduction (FPRED)," is similar to our step 3 (classification). When the site discusses "locations of possible nodules," it is talking about a process similar to what we'll cover in chapter 13.

9.4.4 Downloading the LUNA data

Before we go any further into the nuts and bolts of our project, we'll cover how to get the data we'll be using. It's about 60 GB of data compressed, so depending on your internet connection, it might take a while to download. Once uncompressed, it takes up about 120 GB of space; and we'll need another 100 GB or so of cache space to store smaller chunks of data so that we can access it more quickly than reading in the whole CT.[7]

[7]The cache space required is per chapter, but once you're done with a chapter, you can delete the cache to free up space.

Navigate to https://luna16.grand-challenge.org/download and either register using email or use the Google OAuth login. Once logged in, you should see two download links to Zenodo data, as well as a link to Academic Torrents. The data should be the same from either.

> **TIP** The luna.grand-challenge.org domain does not have links to the data download page as of this writing. If you are having issues finding the download page, double-check the domain for luna16., not `luna.`, and reenter the URL if needed.

The data we will be using comes in 10 subsets, aptly named `subset0` through `subset9`. Unzip each of them so you have separate subdirectories like code/data-unversioned/part2/luna/subset0, and so on. On Linux, you'll need the `7z` decompression utility (Ubuntu provides this via the `p7zip-full` package). Windows users can get an extractor from the 7-Zip website (www.7-zip.org). Some decompression utilities will not be able to open the archives; make sure you have the full version of the extractor if you get an error.

In addition, you need the candidates.csv and annotations.csv files. We've included these files on the book's website and in the GitHub repository for convenience, so they should already be present in code/data/part2/luna/*.csv. They can also be downloaded from the same location as the data subsets.

> **NOTE** If you do not have easy access to ~220 GB of free disk space, it's possible to run the examples using only 1 or 2 of the 10 subsets of data. The smaller training set will result in the model performing much more poorly, but that's better than not being able to run the examples at all.

Once you have the candidates file and at least one subset downloaded, uncompressed, and put in the correct location, you should be able to start running the examples in this chapter. If you want to jump ahead, you can use the code/p2ch09_explore_data.ipynb Jupyter Notebook to get started. Otherwise, we'll return to the notebook in more depth later in the chapter. Hopefully your downloads will finish before you start reading the next chapter!

9.5 Conclusion

We've made major strides toward finishing our project! You might have the feeling that we haven't accomplished much; after all, we haven't implemented a single line of code yet. But keep in mind that you'll need to do research and preparation as we have here when you tackle projects on your own.

In this chapter, we set out to do two things:

1. Understand the larger context around our lung cancer-detection project
2. Sketch out the direction and structure of our project for part 2

If you still feel that we haven't made real progress, please recognize that mindset as a trap—understanding the space your project is working in is crucial, and the design

work we've done will pay off handsomely as we move forward. We'll see those dividends shortly, once we start implementing our data-loading routines in chapter 10.

Since this chapter has been informational only, without any code, we'll skip the exercises for now.

9.6 Summary

- Our approach to detecting cancerous nodules will have five rough steps: data loading, segmentation, grouping, classification, and nodule analysis and diagnosis.
- Breaking down our project into smaller, semi-independent subprojects makes teaching each subproject easier. Other approaches might make more sense for future projects with different goals than the ones for this book.
- A CT scan is a 3D array of intensity data with approximately 32 million voxels, which is around a million times larger than the nodules we want to recognize. Focusing the model on a crop of the CT scan relevant to the task at hand will make it easier to get reasonable results from training.
- Understanding our data will make it easier to write processing routines for our data that don't distort or destroy important aspects of the data. The array of CT scan data typically will not have cubic voxels; mapping location information in real-world units to array indexes requires conversion. The intensity of a CT scan corresponds roughly to mass density but uses unique units.
- Identifying the key concepts of a project and making sure they are well represented in our design can be crucial. Most aspects of our project will revolve around nodules, which are small masses in the lungs and can be spotted on a CT along with many other structures that have a similar appearance.
- We are using the LUNA Grand Challenge data to train our model. The LUNA data contains CT scans, as well as human-annotated outputs for classification and grouping. Having high-quality data has a major impact on a project's success.